
Train Large Models Across the World: Accelerating Training on Geo-Distributed Heterogeneous Clusters via *Subservers* and Bandwidth-Aware Collective Communication

CS650 Project Final Report

Yuxiang Lin

Tianjun Yuan

Abstract

Next-generation foundation models are expected to break the 100 T-parameter barrier, demanding over 10^{25} FLOPs—well beyond the power and network limits of any single GPU cluster. We argue that *geo-distributed* training will be required. With a real-world cluster distribution consideration, we introduce the **subserver** topology: each metropolitan cluster talks to a nearby *subserver* over a multi-terabit bandwidth Internet link, while subservers exchange gradients and activations. We develop a two-tier cost model that (1) chooses pipeline stages across subservers and (2) mixes data *and* tensor sharding inside every cluster, analysing the trade-off. We further apply a full OMNeT++ simulation on eight AWS regions with several clusters in each region.

1 Introduction

Training large language models has outgrown the capabilities of a single server, prompting a shift towards collaborative training across multiple datacenters. According to scaling laws, the next generation of frontier models may require hundreds of trillions of parameters [8]. Even hyperscale infrastructures with $10\mu\text{s}$ switch latencies are constrained by limits on power, cooling, and fiber optics, making it infeasible to host such vast models within a single metropolitan area.

We propose a subserver-centric training framework that adapts the parallelism strategy to each level of a heterogeneous, geo-distributed network. Specifically, we adopt:

- Pipeline parallelism across subservers,
- Data and tensor parallelism within each metro area to fully utilize high-bandwidth fabrics such as Jupiter.

Using a compact α - β communication model, we demonstrate conditions under which the added cost of local collective operations in tensor sharding is outweighed by the reduction in global communication time.

Contributions.

- **Subserver topology:** A practical relay-layer architecture that efficiently aggregates metro-area traffic.
- **Unified scheduling model:** A parallelism-aware scheduler that jointly optimizes pipeline, data, and tensor strategies across heterogeneous clusters.
- **Tensor parallelism trade-off analysis:** A closed-form bound to determine whether sharding model weights leads to faster training for a given layer or parameter configuration.
- **End-to-end simulation:** An OMNeT++-based prototype simulating geo-distributed training across eight AWS regions, each containing multiple compute clusters.

2 Background and Motivation

2.1 Collective Communication Primitives & Parallelism Taxonomy

Concepts. Large-batch SGD relies on three bandwidth-hungry collectives:

- (i) *ReduceScatter* splits a tensor across P ranks and simultaneously sums corresponding slices
- (ii) *AllGather* concatenates those slices so every rank recovers the full tensor
- (iii) *AllReduce* is simply a *ReduceScatter* followed by an *AllGather*.

On top of these kernels, four orthogonal forms of parallelism are routinely combined to scale a deep network [3]:

- **Data parallelism:** replicate parameters, shard the mini-batch,
- **Fully-sharded data parallelism:** shard both activations and weights,
- **Tensor/model parallelism:** partition large weight matrices within a layer,
- **Pipeline parallelism:** place consecutive layer groups on different devices.

Hybrid schemes such as HDP/HPP [22] or SDPipe’s grid layout [13] exploit a subset of these axes but typically ignore WAN latency and heterogeneous link rates [12].

2.2 Decentralized Large Model Training

With the rapid growth in the size of foundation models [1, 20], training large-scale neural networks on a single machine has become increasingly impractical. *Distributed learning* [5] tackles this challenge by splitting the computational and data loads across multiple nodes, alleviating resource bottlenecks. Modern large-scale training clusters are often heterogeneous, comprising CPUs, GPUs, and specialized accelerators [6].

However, looking ahead, scaling laws and recent capacity studies indicate that the next generation of foundation models may reach ~ 100 T parameters—much larger than today’s GPT-4-class systems [8, 1]. Training such a model would demand on the order of 10^{25} FLOPs and sustained multi-gigawatt power draw, far beyond the logistical limits of any single metropolitan region [8]. To unlock this level of compute, we advocate *geo-distributed* training that federates clusters on different continents, effectively creating a “planet-scale” accelerator fabric whose aggregate capacity can absorb the required workload [23].

Internet speed and latency also plays a significant role in *geo-distributed* training. Hyperscalers have begun to deploy RDMA-over-Ethernet (RoCE v2) backbones that interconnect *tens of thousands* of GPUs with $<10\mu\text{s}$ hop latency, demonstrated in Meta’s 2024 clusters [18, 7]. Upcoming photonic interposers such as Lightmatter’s *Passage* also promise $10\text{--}100\times$ higher bandwidth at a fraction of the power, enabling million-GPU topologies [4]. Moreover, algorithms like *NetStorm* dynamically reconfigure gradient-exchange trees to exploit heterogeneous, fluctuating wide-area links, achieving up to $9\times$ speed-ups over conventional parameter servers [10]. Taken together, these trends point to a near-future in which multi-gigabit transoceanic links, elastic optical fabrics, and adaptive communication schedules converge, allowing synchronized SGD across continents and, ultimately, the training of ‘hundred times’ models on a truly global pool of computation.

2.3 Decentralized Training with Subservers

Urban clusters & regional hubs. Empirically, metro-area “GPU cities” already exhibit $<150\mu\text{s}$ round-trip RTT between hosts—e.g. Google Cloud’s own intra-zone measurements report a $\sim 66\mu\text{s}$ mean latency on 100 GbE links. Inside such a area, fifth-generation Jupiter fabrics now deliver up to 13 Pbit/s of bisection bandwidth, implying per-rack aggregates well above 5 Tbit/s [21].

By contrast, distances between major cities stretch thousands of kilometres; end-to-end RTTs still range from ~ 10 ms to 60 ms within continents and hover around 60 ms across the Planet [24].

This issue in both *bandwidth* and *latency* motivates the introduction of a regional **subserver** that aggregates all local traffic and pipelines to a regional center (Fig. 1).

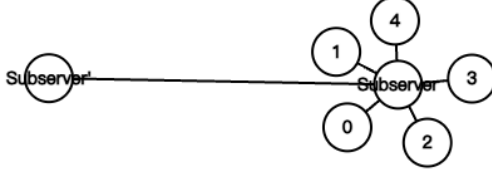


Figure 1: Topology with *Subserver*

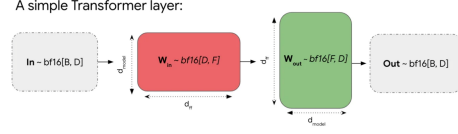


Figure 2: A Transformer layer simplified to two matrix multiplications [3, Part 5].

Link heterogeneity in the subserver topology.

- **Cluster $\rightarrow$ Subserver**: multi-Terabit electrical or short-reach optical fabric, $\text{RTT} \leq 0.15$ ms
- **Subserver $\leftrightarrow$ Subserver** (“metro $\leftrightarrow$ metro”): long-haul coherent optics at 400–800 Gb/s per lambda, $\text{RTT} \approx 10$ ms to 60 ms. However, large bandwidth between Subserver significantly helps with our data transmitting.

3 Problem Formulation

3.1 Simplified Layer and Cost Model

We model each Transformer block as two dense matmuls: $\text{In}[B, D] = X \xrightarrow{\cdot} W_{\text{in}}[D, F] \xrightarrow{\cdot} W_{\text{out}}[F, D]$ (Fig. 2). Element-wise ops are ignored because their cost is small compared with the $\Theta(BD^2)$ multiplies.

Communication follows the α – β rule: sending n bytes costs $\alpha + \beta n$. Local links share $(\alpha_{\text{loc}}, \beta_{\text{loc}})$, while long-haul links between Subservers share $(\alpha_{\text{wan}}, \beta_{\text{wan}})$ with $\alpha_{\text{wan}} \gg \alpha_{\text{loc}}$. Compute and communication overlap whenever their critical paths do not clash. Our objective is to pick a parallelism plan that minimises the end-to-end makespan on this two-tier network.

3.2 Parallelism on the Subserver Topology

- **Subserver & Subserver** (10 to 60 ms RTT): use *pipeline parallelism*. Each stage streams activations forward and gradients back, hiding most link latency behind compute [9, 14].
- **Cluster & Subserver** (≤ 0.15 ms RTT): combine *data parallelism* (split the mini-batch) and *tensor/model parallelism* (shard weight matrices). Fast fabrics make collectives such as AllReduce negligible [19, 17].

Streaming coarse blocks over slow links and keeping fine-grained synchronous work inside fast links yields better utilisation than relying on any single parallelism mode end-to-end.

4 Analysis of Parallelism Approaches

4.1 Notations

We use the same notation as in [3, Part 3] to represent a sharded tensor, which is a tensor divided along some axes and distributed among devices. $A[I, J]$ represents a tensor of size $I \times J$. $A[I_X, J_Y]$ is A with I -dimension sharded along the X -axis, and J -dimension sharded along the Y -axis.

When multiplying two matrices along the same sharded axis in parallel, the results distributed among the devices need to be summed to get the final result. The summing operation is an AllReduce, which

Symbol	Meaning
L	Number of layers in the model
D	d_{model} (hidden dimension)
F	d_{ff} (feed-forward dimension)
B	micro-batch size
n, m_i	n clusters, i -th cluster has m_i clients
i, j	The j -th client in i -th cluster
$b_{i,j}$	Per-device micro-batch size
$(u), (d)$	Upload/download
$c_{i,j}$	Compute FLOPS
$v_{i,j}, w_{i,j}$	Intra-cluster link latency and bandwidth
$V_{i_1, i_2}, W_{i_1, i_2}$	Inter-cluster link latency and bandwidth

Figure 3: A table of symbols used and their meanings.

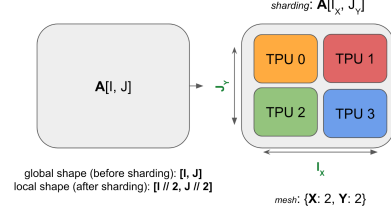


Figure 4: Sharding the tensor A of size $I \times J$ over the X and Y axes respectively. Each device (TPU in this example) holds a piece of the sharded tensor.

could also be decomposed into a ReduceScatter followed by an AllGather. For example:

$$\begin{aligned}
A[I, J_X] \cdot_J B[J_X, K] &\rightarrow C[I, K] \{U_X\} \\
\text{AllReduce}_X C[I, K] &\rightarrow C[I, K] \\
\text{ReduceScatter}_{X,K} C[I, K] \{U_X\} &\rightarrow C[I, K_X] \\
\text{AllGather}_X C[I, K_X] &\rightarrow C[I, K]
\end{aligned}$$

4.2 Pipeline Parallelism

Pipeline parallelism splits model layers across devices. It is useful for overcoming the memory constraint of individual devices and it is less communication intensive than the other parallelism approaches.

To prevent pipeline bubbles when using pipeline parallelism, multiple “micro-batches” are sent down the pipeline before gradient update for the first micro-batch returns to the first layer. This will have a small negative impact on convergence due to using stale values. However, since we are unable to find a paper with analytical bounds for convergence when using micro-batching, we would assume that our approach would have a rate of convergence close to similar approaches such as [22].

In our approach, pipeline parallelism is used across clusters, as the less communication intensive nature of pipeline parallelism is suitable for the high latency, high bandwidth links between the clusters. It also addresses the heterogeneity of memory resources among clusters by assigning more model layers to clusters with larger memory. However, this paper does not focus its analysis on the memory constraints.

Let π_n be a permutation of $\{1, 2, \dots, n\}$, the inter-cluster communication cost without consider pipelining is

$$\sum_{i=1}^{n-1} \left(V_{\pi_i, \pi_{i+1}} + \frac{BD}{W_{\pi_i, \pi_{i+1}}} \right). \quad (1)$$

It is difficult to formulate the speedup caused by pipelining since the micro-batches approach each cluster at different times. Additionally, a different number of layers may be assigned to each cluster. However, suppose that the number of micro-batches is high, and that the layers are evenly assigned, the speedup of n clusters in a pipeline can be approximated as a factor of n .

4.3 Data Parallelism

Sharding Scheme:

$$\text{In}[B_X, D] \cdot_D W_{\text{in}}[D, F] \cdot_F W_{\text{out}}[F, D] \rightarrow \text{Out}[B_X, D]$$

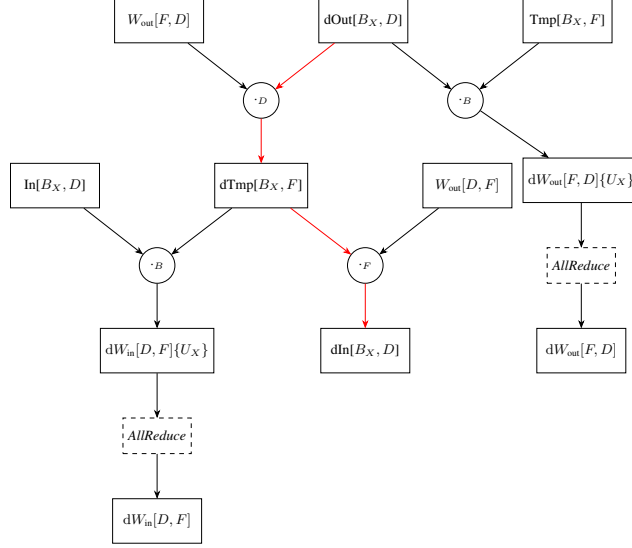


Figure 5: Data flow diagram for the backward pass of data parallelism. The critical path that computes dIn from dOut is colored in red.

Data parallelism [3, Part 5] shards the input data along the batch dimension. The forward pass consists of two steps,

$$\begin{aligned} \text{In}[B_X, D] \cdot_D W_{\text{in}}[D, F] &\rightarrow \text{Tmp}[B_X, F] \\ \text{Tmp}[B_X, F] \cdot_F W_{\text{out}}[F, D] &\rightarrow \text{Out}[B_X, D], \end{aligned}$$

which could be computed by each node on their own without communication. The backward pass shown in 5 requires communication between nodes to compute the gradient of model weights. However, the critical path from dOut to dIn, which computes the gradient of the input activation, does not require communication between nodes. Therefore, the computation of weight gradients in one layer will not bottlenecked by the communication time within the previous layer.

Data parallelism is used inside the clusters as it requires the communication of weight gradients at every layer during the backward pass. Since the within-cluster links have both low latency and more throughput, it is more advantageous to perform an operation that communicates frequently within as opposed to between clusters.

For each micro-batch, the per-cluster compute cost of forward and backward pass is

$$T_{\text{compute},i} = 2 \max_{0 \leq j < m_i} \left\{ \frac{2b_{i,j}DFl_i}{c_{i,j}} \right\}. \quad (2)$$

Communication cost for single-layer gradient exchange (ReduceAll) can be modeled by

$$T_{\text{communication},i} = 2 \max \left\{ \max_{1 \leq j < m_i} \left\{ v_{i,j,(u)} + \frac{DF}{w_{i,j,(u)}} \right\}, \max_{1 \leq j < m_i} \left\{ v_{i,j,(u)} + v_{i,j,(d)} + \frac{DF}{w_{i,j,(d)}} \right\} \right\}, \quad (3)$$

where the reduction happens at the subserver as each client uploads, and that the result of the reduction of one chunk is downloaded following the upload of that chunk in pipeline as soon as the reduction is complete. However, note that communication cost is determined by the slowest link in a cluster and that DF amount of data is always exchanged regardless of the partitioning scheme. Thus, data parallelism cannot be used to address the communication heterogeneity among the clients in a cluster.

Furthermore, the gradient exchange operation does not need to occur at the backward pass of every single micro-batch, but can be deferred until the end of the entire mini-batch, as illustrated in Figure 6. This is due to that the micro-batches are effectively a partitioning of the mini-batch where instead of sending down the entire mini-batch at once, it is sent micro-batch by micro-batch, and that every micro-batch in the same mini-batch is multiplied by the same weight.

Cluster \ Time	0	1	2	3	4	5	6
0	0F	1F			0B	1B	ReduceAll
1		0F	1F	0B	1B	ReduceAll	
2			0F	1F	ReduceAll		

Figure 6: Example of inter-cluster pipeline parallelism and inter-cluster data parallelism with three clusters, and two micro-batches in a single batch.

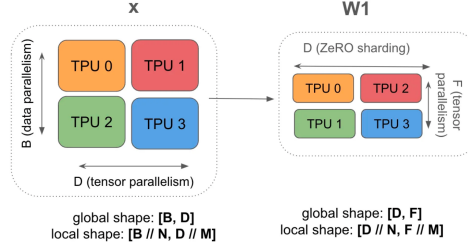


Figure 7: Tensor parallelism by sharding over both X and Y dimensions.

4.4 Tensor Parallelism

Sharding Scheme:

$$\text{In}[B_X, D_Y] \cdot_D \text{W}_{\text{in}}[D_X, F_Y] \cdot_F \text{W}_{\text{out}}[F_Y, D_X] \rightarrow \text{Out}[B_X, D_Y] \quad (4)$$

Tensor Parallelism (Forward & Backward)

```

Forward pass (compute Loss[BX])
In[BX, D] = AllGather_Y(In[BX, DY]) (critical)
Win[D, FY] = AllGather_X(Win[DX, FY]) (prefetch)
Tmp[BX, FY] = In[BX, D] ·D Win[D, FY]
Wout[FY, D] = AllGather_X(Wout[FY, DX]) (prefetch)
Out[BX, D]{UY} = Tmp[BX, FY] ·F Wout[FY, D]
Out[BX, DY] = ReduceScatter_Y(Out[BX, D]{UY}) (critical)
Loss[BX] = ...

```

```

Backward pass (dWout[FY, DX], dWin[DX, FY])
dOut[BX, D] = AllGather_Y(dOut[BX, DY]) (critical)
dWout[FY, D]{UX} = Tmp[BX, FY] ·B dOut[BX, D]
dWout[FY, DX] = ReduceScatter_X(dWout[FY, D]{UX})
Wout[FY, D] = AllGather_X(Wout[FY, DX]) (prefetch)
dTmp[BX, FY] = dOut[BX, D] ·D Wout[FY, D]
In[BX, D] = AllGather_Y(In[BX, DY]) (shared)
dWin[D, FY]{UX} = dTmp[BX, FY] ·B In[BX, D]
dWin[DX, FY] = ReduceScatter_X(dWin[D, FY]{UX})
Win[D, FY] = AllGather_X(Win[DX, FY]) (prefetch)
dIn[BX, D]{UY} = dTmp[BX, FY] ·F Win[D, FY]
dIn[BX, DY] = ReduceScatter_Y(dIn[BX, D]{UY}) (critical)

```

To address the issue with pure data parallelism, we consider tensor parallelism, which shards weights over devices. Sharding weights over the X axis is sometimes referred to as fully-sharded data parallelism (FSDP), and sharding over the Y axis is sometime simply called tensor parallelism [3, Part 5]. However, we focus on sharding over both the X and Y axis, since sharding over only one axis will not be helpful in reducing the communication cost of each AllGather/ReduceScatter operation. This is due to, for example, when sharding a tensor of size IJ over n nodes in just the X axis, the amount of data send/received by each node is still $\frac{m-1}{m}IJ$.

Let $m_{i,X}$ and $m_{i,Y}$ be the number of nodes in the i -th cluster along the X – and Y –axis of sharding, respectively, such that $m_{i,X}m_{i,Y} = m_i$. Let $p_{i,(j_X,j_Y)}$ be the relative size of the tensor shard assigned to node (j_X, j_Y) in the i -th cluster, such that $\sum_{j_X} \sum_{j_Y} p_{i,(j_X,j_Y)} = 1$.

Our analysis refers to the pseudo-code in *Tensor Parallelism (Forward & Backward)* 4.4. The formula for the the communication time costs are complicated. For example, the time it takes to perform $\text{AllGather}_Y([B_X, D_Y])$ is

$$\max_{0 \leq j_X < m_{i,X}} \left\{ \max \left\{ \max_{0 \leq j_Y < m_{i,Y}} \left\{ v_{i,(j_X,j_Y),(u)} + \frac{p_{i,(j_X,j_Y)}BD}{w_{i,(j_X,j_Y),(u)}} \right\}, \right. \right. \\ \left. \left. \max_{0 \leq j_Y < m_{i,Y}} \left\{ v_{i,(j_X,j_Y),(d)} + \frac{\sum_{j'_Y \neq j_Y} p_{i,(j_X,j'_Y)}BD}{w_{i,(j_X,j_Y),(d)}} \right\} \right\} \right\}.$$

To simplify our analysis, we approximate the communication cost along the Y -axis with

$$v + \frac{BD}{\min m_{i,X}w}, \quad (5)$$

which assumes uniform latency of v and bandwidth of w . The communication cost along the X -axis can also be approximated with

$$v + \frac{FD}{\min m_{i,Y}w}. \quad (6)$$

It can be observed that the overall communication cost is reduced. However, the communication cost, on the other hand, is increased by

$$2 \left(v + \frac{BD}{\min m_{i,X}w} \right) \quad (7)$$

on top the original communication cost of data parallelism. Therefore, applying tensor parallelism would be beneficial when BD is small.

5 Evaluation

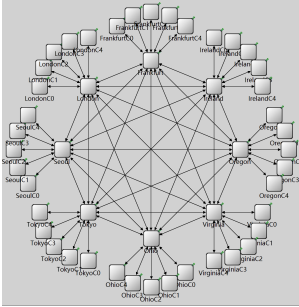


Figure 8: The topology used in simulation with clusters located in Ohio, Virginia, Oregon, Ireland, Frankfurt, London, Seoul, and Tokyo AWS regions.

Parameter	Value
L (# layers in the model)	126
D (hidden dimension)	16384
F (feed-forward dimension)	53248
$\mathcal{N}$ (# input tokens)	10^9
$\mathcal{B}$ (batch size)	10^6
n (# clusters)	8
m_i (# clients in clusters)	5
$v_{i,j}, w_{i,j}$	(median consumer link in country)
$V_{i,j}, W_{i,j}$	(AWS instance benchmark)

Figure 9: The parameters used in the simulation.

We use OMNeT++ [11], a modular, component-based C++ simulation library and framework, to measure the simulated training time on a representative network ¹.

As shown in Figure 8, The representative framework consists of eight geo-distributed clusters with one subserver and five clients per cluster. We use the network latency and bandwidth between eight

¹Code available at <https://github.com/theta-lin/cs-650-project-code>.

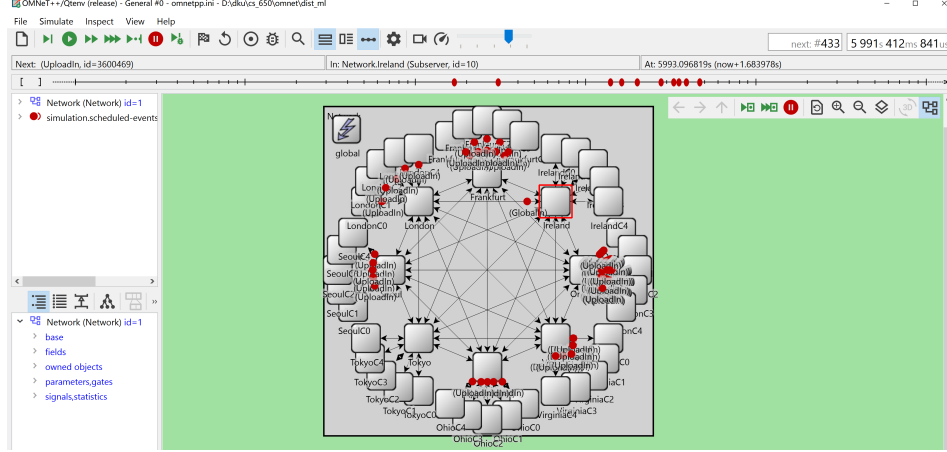


Figure 10: Simulation in progress.

B (micro-batch size)	Time to train (T_{actual} in days)	FLOPS utilization
100,000	581.36	2.43×10^{-4}
50,000	476.70	2.97×10^{-4}
10,000	452.70	3.12×10^{-4}

Figure 11: Simulation result with fixed client FLOPS $c_{i,j} = 3 \times 10^{14}j$ and varying micro-batch size B .

AWS instances distributed in eight AWS regions [23, Sec. D.3] for the inter-cluster latency and bandwidth. As for the intra-cluster latency and bandwidth, we use the median fixed broadband latency and bandwidth from Speedtest Global Index [16] in the country where the AWS region of the corresponding subserver is located, which is representative of the consumer Internet link characteristics in different countries.

The parameters used in the simulation is shown in Table 9. For the parameter sizes of the model being trained, we use the parameter sizes of Llama 3.1 405B [2], which is representative of a large model that may require distributed training.

The strategy simulated in the experiment consists of pipeline parallelism between clusters and data parallelism within clusters. For simplicity, model layers are evenly distributed among clusters ($\frac{L}{n}$ layers per cluster), and input tokens are also evenly distributed among clients within clusters ($\frac{B}{m_i}$ tokens per client). Note that this simplification disregards the possible heterogeneity of total memory between clusters and did not adapt for the heterogeneity of the compute power between clients. The heterogeneity of inter-cluster link characteristics is exploited by enumerating all the permutations of the subservers and search for the permutation with the least total inter-cluster communication time before the simulation.

FLOPS utilization is used as a metric for the efficiency of the communication scheme. It can be calculated with:

$$\text{FLOPS utilization} = \text{actual FLOPS} \cdot \frac{1}{\text{total client FLOPS}} = \frac{4 \cdot \mathcal{NDFL}}{T_{\text{actual}}} \frac{1}{\sum_{i=0}^{n-1} \sum_{j=0}^{m_i-1} c_{i,j}} \quad (8)$$

We first investigate the impact of micro-batch size on training performance. For simplicity, the client FLOPS's are set to $c_{i,j} = 3 \times 10^{14}(j + 1)$ across all clusters. This is to roughly match NVIDIA A100 80GB PCIe Tensor Core's 3.12×10^{14} FLOPS BFLOAT16 compute performance [15]. The results are shown in Table 11. Note that FLOPS utilization is very low, indicating that the training speed is bottlenecked by the network communication cost.

To further investigate the impact of network communication on training performance, we fix micro-batch size to $B = 50,000$ and varies the client FLOPS. Results in Table 12 indicates that the FLOPS utilization indeed improves as client FLOPS decreases. This indicates that in our representative net-

$c_{i,j}$ (client FLOPS)	Time to train (T_{actual} in days)	FLOPS utilization
$3 \times 10^{12}(j+1)$	483.35	2.92×10^{-2}
$3 \times 10^{13}(j+1)$	477.22	2.96×10^{-3}
$3 \times 10^{14}(j+1)$	476.70	2.97×10^{-4}

Figure 12: Simulation result with fixed micro-batch size $B = 50,000$ and varying client FLOPS $c_{i,j}$.

work topology, the training time is highly dominated by the network communication time. Therefore, such a distributed training scheme might not be feasible in the current Internet infrastructure.

6 Related Work

Multi-dimensional parallelism. Early systems such as GPipe introduced inter-batch *pipeline parallelism* to hide WAN-scale latencies for very deep models [9]. PipeDream generalised this idea with weight stashing and micro-batching, achieving up to $5.3\times$ speed-ups over pure data parallelism [14]. On the intra-layer axis, Megatron-LM pioneered coarse-grained *tensor parallelism* that shards each matrix across multiple GPUs while keeping communication local [19]. Our work not only combines these dimensions, but crucially adds a third geo-hierarchical tier (the subserver) that was absent from earlier designs.

Geo-distributed training. [23] introduces geo-distributed machine learning for large language models. However, they haven’t consider subserver or tensor parallelism analysis. Recent methods such as NetStorm dynamically reshaping spanning-tree topologies to match fluctuating WAN links and achieves $6.5\text{--}9.2\times$ faster synchronisation [10]. Our subserver abstraction can be viewed as a pragmatic middle ground: it retains synchronous SGD, but limits long distance transfers to a single per-step pipeline message.

In summary, prior work either optimises one parallel dimension, assumes uniform fabrics, or targets asynchronous geo-ML. We instead provide the first end-to-end analysis of *heterogeneous, synchronous* training that spans metropolitan clusters via subservers.

7 Conclusion & Future Work

We introduced a *subserver-centred* architecture that bridges metro-scale GPU with long-haul optical links and analysed how to layer pipeline, data and tensor parallelism on top of this two-tier network. Moreover, we also conducted simulation on OMNeT++ with eight AWS regions and clusters across these regions to calculate training time, FLOPS utilization.

The next step could be deploying the full stack on a real multi-region testbed and train a 7-to-70 B-parameter Transformer to quantify convergence, energy and cost trade-offs. We believe these steps will move geo-distributed training from simulation to a deployable reality, unlocking the computational headroom required for 100 T-scale frontier models.

ML Tool Use

ML tools including ChatGPT-4o and NotebookLM have been used for researching and summarizing text and to refine and improve the grammar and format of the text and pseudo code.

References

- [1] Josh Achiam et al. “Gpt-4 technical report”. In: *arXiv preprint arXiv:2303.08774* (2023).
- [2] Meta AI. *Llama 3.1 405B*. <https://huggingface.co/meta-llama/Llama-3.1-405B>. Accessed: 2025-04-18. 2025.
- [3] Jacob Austin et al. “How to Scale Your Model”. In: (2025). Retrieved from <https://jax-ml.github.io/scaling-book/>.
- [4] Ke Chen et al. “Graphene photonic crystal fibre with strong and tunable light–matter interaction”. In: *Nature Photonics* 13.11 (2019), pp. 754–759.

- [5] Mingzhe Chen et al. “Distributed learning in wireless networks: Recent progress and future challenges”. In: *IEEE Journal on Selected Areas in Communications* 39.12 (2021), pp. 3579–3605.
- [6] Jiangfei Duan et al. “Efficient training of large language models on distributed infrastructures: a survey”. In: *arXiv preprint arXiv:2407.20018* (2024).
- [7] Adithya Gangidi et al. “Rdma over ethernet for distributed training at meta scale”. In: *Proceedings of the ACM SIGCOMM 2024 Conference*. 2024, pp. 57–70.
- [8] Alexandru M Gherghescu et al. “A Look Into Training Large Language Models on Next Generation Datacenters”. In: *arXiv e-prints* (2024), arXiv–2407.
- [9] Yanping Huang et al. “Gpipe: Efficient training of giant neural networks using pipeline parallelism”. In: *Advances in neural information processing systems* 32 (2019).
- [10] Zonghang Li et al. “Accelerating geo-distributed machine learning with network-aware adaptive tree and auxiliary route”. In: *IEEE/ACM Transactions on Networking* (2024).
- [11] OpenSim Ltd. *The OMNeT++ Discrete Event Simulation System*. <https://omnetpp.org/>. Accessed: 2025-04-18. 2001.
- [12] Ziyue Luo et al. “Efficient pipeline planning for expedited distributed dnn training”. In: *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE. 2022, pp. 340–349.
- [13] Xupeng Miao et al. “Sdpipe: A semi-decentralized framework for heterogeneity-aware pipeline-parallel training”. In: *Proceedings of the VLDB Endowment* 16.9 (2023), pp. 2354–2363.
- [14] Deepak Narayanan et al. “PipeDream: Generalized pipeline parallelism for DNN training”. In: *Proceedings of the 27th ACM symposium on operating systems principles*. 2019, pp. 1–15.
- [15] NVIDIA Corporation. *NVIDIA A100 Tensor Core GPU Datasheet*. <https://www.nvidia.com/en-us/data-center/a100/>. Accessed: 2025-04-18. 2020.
- [16] Ookla. *Speedtest Global Index*. <https://www.speedtest.net/global-index>. Accessed: 2025-04-18. 2025.
- [17] Adam Roberts et al. “Scaling up models and data with t5x and seqio”. In: *Journal of Machine Learning Research* 24.377 (2023), pp. 1–8.
- [18] Allyn Romanow and Stephen Bailey. “An Overview of RDMA over IP”. In: *Proceedings of the First International Workshop on Protocols for Fast Long-Distance Networks (PFLDnet 2003)*. Citeseer. 2003.
- [19] Mohammad Shoeybi et al. “Megatron-lm: Training multi-billion parameter language models using model parallelism”. In: *arXiv preprint arXiv:1909.08053* (2019).
- [20] Hugo Touvron et al. “Llama: Open and efficient foundation language models”. In: *arXiv preprint arXiv:2302.13971* (2023).
- [21] A. Vahdat. *Speed, scale and reliability: 25 years of Google data-center networking evolution*. 2024. URL: <https://cloud.google.com/blog/products/networking/speed-scale-reliability-25-years-of-data-center-networking>.
- [22] Shengyuan Ye et al. “Asteroid: Resource-efficient hybrid pipeline parallelism for collaborative DNN training on heterogeneous edge devices”. In: *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 2024, pp. 312–326.
- [23] Binhang Yuan et al. “Decentralized training of foundation models in heterogeneous environments”. In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 25464–25477.
- [24] M. Zacchello. *What is Latency, and How Can You Address It?* 2023. URL: <https://blog.equinix.com/blog/2023/03/01/what-is-latency-and-how-can-you-address-it/>.